

Smart College Attendance System - Team Work Plan

Smart College Attendance System — Team Work Plan

Geofencing + Face Recognition + Liveness Detection

1. Project Overview

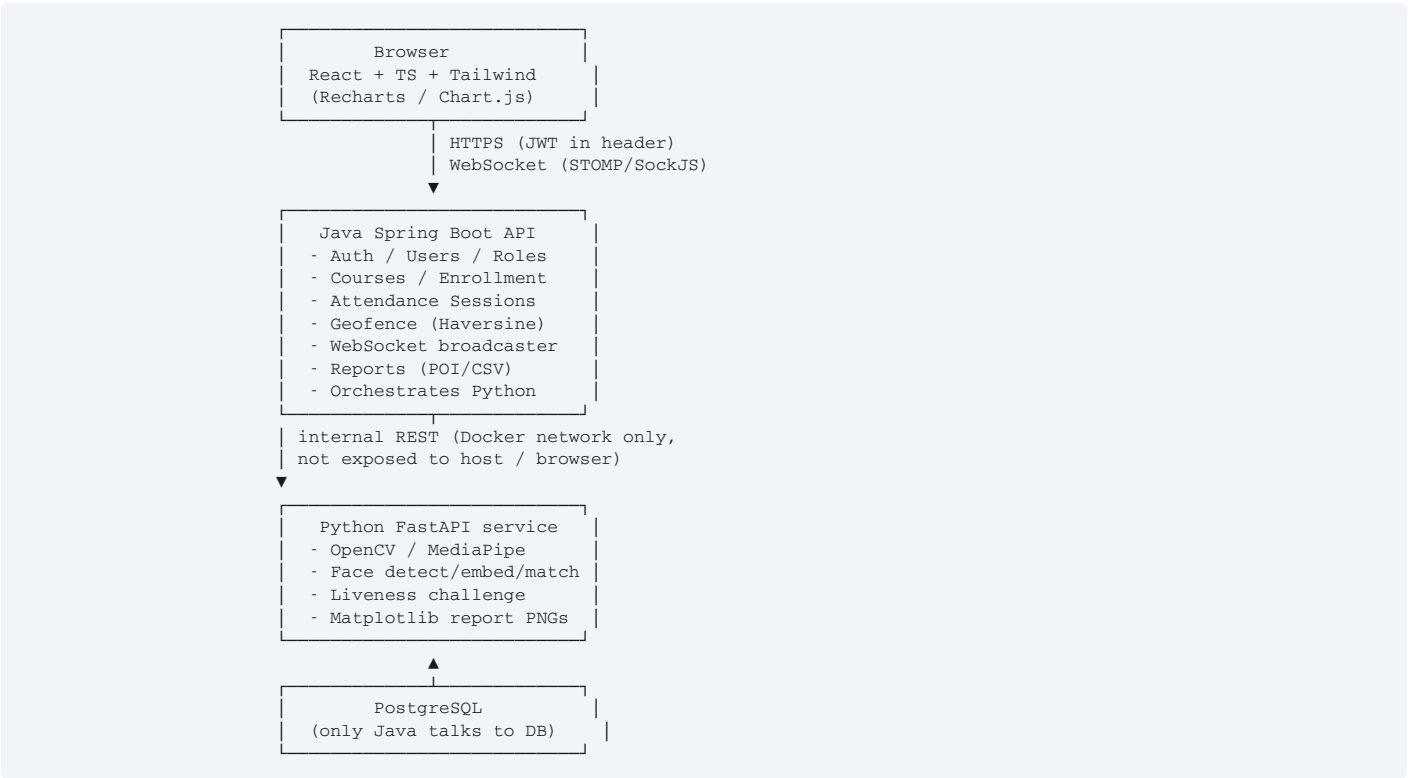
This is a college-scale attendance platform where a student's presence is confirmed through **three independent checks** stacked together: GPS geofencing (is the student physically near the classroom?), face verification (is this the enrolled student?), and liveness detection (is this a live person, not a photo/video replay?). Only when all three pass does Java mark attendance and push a live update to the teacher's screen over WebSocket.

The project is deliberately split along a **hard technical boundary that already matches your team's skills**:

- Anything that is "business logic + database + security" → **Java Spring Boot** (Members 1 & 2, both Java devs).
- Anything that is "computer vision / ML" → **Python FastAPI**, called only by Java, never by the browser (Members 3 & 4-partial, both Python devs).
- Anything that is "what the user sees" → **React + TypeScript**, owned mainly by Member 4, since dashboards are UI work, not Python work.

This document treats that boundary as fixed and designs everything else — schema, APIs, timeline, testing, docs — around it.

2. Final Architecture



Key rule enforced everywhere in this plan: **the browser only ever talks to Spring Boot**. The FastAPI container has no port published to the outside world in `docker-compose.yml` — only Spring Boot's network alias can reach it.

3. Technology Stack (as required, nothing extra)

Layer	Technology	Owner
Frontend	React, TypeScript, Tailwind CSS	Member 4 (primary)
Charts (dashboard)	Recharts / Chart.js	Member 4
Backend (public)	Java Spring Boot	Members 1 & 2
Backend (internal AI)	Python FastAPI	Members 3 & 4 (liveness only)
DB	PostgreSQL	Member 1 (schema owner)
Auth	Spring Security + JWT	Member 1
CV/AI	OpenCV, MediaPipe, face embeddings	Member 3
Liveness	MediaPipe (blink/head-turn) in Python	Member 4
Geolocation	Browser Geolocation API + Haversine	Member 4 (UI) + Member 2 (calc)
Analytics (report graphs)	Matplotlib	Member 3 (data) + Member 4 (integration)
Real-time	WebSocket (STOMP over SockJS)	Member 2
Reports	Apache POI (Excel), CSV	Member 2 (+ Member 1 utility)
Deployment	Docker, Docker Compose	Member 4 (+ Member 3 for Python image)
VCS	Git + GitHub	All

No Redis, no Kafka, no microservice mesh, no Kubernetes — a college project with 2 backend services and a frontend does not need them. This is called out again in §17.

4. Detailed Role of Member 1 — Java Developer A ("Core Platform")

Mission: Own everything that exists *before* a session even starts — identity, structure, and data model. Everyone else's work sits on top of your tables and your auth.

Primary ownership:

- PostgreSQL schema design (all tables in §9) — you write the source-of-truth Flyway/Liquibase migration scripts.
- Spring Boot project skeleton: package structure, layered architecture (controller/service/repository/dto), global exception handler (`@ControllerAdvice`), standard API response wrapper.
- User, Student, Teacher entities + repositories + services.
- Course entity + CRUD service.
- Enrollment entity + service (enroll/remove/list students in a course).
- Spring Security config, JWT filter, token generation/validation, refresh-token handling (optional).
- Role-based authorization (`STUDENT` , `TEACHER` , `ADMIN`) via method-level `@PreAuthorize` .
- DTO + validation layer (Bean Validation annotations) used as the pattern for Member 2's DTOs too.
- BCrypt password hashing.
- Common Java utility layer used by others: `ApiResponse<T>` , `PageResponse<T>` , custom exceptions (`ResourceNotFoundException` , `DuplicateAttendanceException` , etc.), a shared `ExcelExportUtil` class *skeleton* that Member 2 extends for attendance-specific reports (this removes Excel/CSV plumbing work from Member 2's plate — see rebalancing in §8).

Secondary (rebalancing) tasks:

- Co-author the Postman collection for auth/course/enrollment endpoints.
- Own the SRS and Installation Guide documentation (§19).
- Write JUnit tests for auth, user, course, enrollment modules.

What you are **not** responsible for: geofence math, WebSocket, Python integration, face/liveness, React UI. You hand Member 2 a clean `User / Student` entity and a working JWT filter to build on.

5. Detailed Role of Member 2 — Java Developer B ("Attendance Engine")

Mission: Own the moment attendance actually happens — sessions, geofencing, orchestration of Python, real-time updates, and reporting.

Primary ownership:

- `AttendanceSession` entity + service: create/start/end session, set location (lat/lng), radius (meters), start/end time window.
- Geofence verification service: validates incoming lat/lng/accuracy from the browser, runs the **Haversine formula** against the session's stored center point and radius, returns pass/fail + distance.
- Attendance business logic: the single service method that sequences "geofence OK → call Python liveness → call Python face-match → apply thresholds → write `attendance` row" — this is the most important piece of glue code in the whole project.

- Java → Python HTTP client (`RestTemplate` / `WebClient`) calling the internal FastAPI endpoints defined in §11, including timeout/retry handling and mapping Python error responses to Java exceptions.
- WebSocket configuration (STOMP + SockJS), broadcasting `AttendanceMarkedEvent` to a per-session topic (e.g. `/topic/session/{id}`) so the teacher dashboard updates instantly.
- Attendance reports: CSV export + Excel export using Apache POI, built on top of Member 1's `ExcelExportUtil` skeleton.
- Security hardening: CORS configuration, request rate limiting (e.g. Bucket4j) or a simple interceptor) on the verification endpoints (these are the most sensitive/expensive calls).
- Integration testing across the full session→verification→attendance chain.

Secondary tasks:

- Pair with Member 3 early (Week 1–2) to **freeze the Python API contract** (§11) since your orchestration code depends on it directly.
- Contribute to the Database Documentation (schema rationale for session/attendance/verification tables, since you own those tables' behavior).

Why this split is fair despite looking "bigger": Member 1's work is broad-but-shallow (many simple CRUD modules); Member 2's is narrow-but-deep (one complex flow). Total effort is designed to be comparable — see §22 balance check.

6. Detailed Role of Member 3 — Python Developer A ("Face AI Core")

Mission: Own the actual computer-vision decision: "is this the enrolled student's face, captured live from the camera, with acceptable quality?"

Primary ownership:

- FastAPI project skeleton for the internal microservice (routers, Pydantic schemas, settings via `.env`).
- Face detection using OpenCV/MediaPipe (locate face bounding box, reject frames with zero or multiple faces).
- Face quality checks: blur detection (variance of Laplacian), brightness/lighting check, face size/pose check — reject bad frames before wasting an embedding computation.
- Face embedding generation (e.g., using a pretrained embedding model such as `face_recognition` /dlib-based encodings, or a lightweight ONNX/FaceNet model — pick one early, see Risk #1 in §18).
- Face enrollment pipeline: when a student registers their face (usually once), generate and store the embedding.
- Face comparison/verification: compute distance (cosine/Euclidean) between the stored embedding and the live one, apply a tuned threshold, return match/no-match + confidence score.
- Threshold testing/tuning using a small sample dataset (your own team's faces, different lighting).
- ML-focused unit tests: known-good pair should match, known-different pair should not, corrupted/blank image should be rejected gracefully.

Secondary (rebalancing) tasks:

- Prepare the aggregated **data** for Matplotlib analytics (e.g., verification confidence score distribution, match/no-match counts per session) as simple functions/DataFrames — Member 4 then wires these into the report endpoint and renders/serves the PNGs. This splits the "Matplotlib" line item fairly between the two Python devs instead of dumping it entirely on Member 4.
- Write the Python service's `Dockerfile` (you know its exact dependency footprint — OpenCV/dlib installs are notoriously fiddly, so the person who fought the dependencies should own the image).
- Contribute to ML Methodology documentation (§19).

What you are **not** responsible for: liveness challenge logic (blink/head-turn — that's Member 4's), React, geofencing, database schema.

7. Detailed Role of Member 4 — Python Developer B ("Liveness + Frontend")

Mission: Two very different jobs, split cleanly: (a) the Python liveness check, and (b) essentially the entire React frontend. This is intentional — you're the bridge between the AI backend and the human using it.

Primary ownership — Python:

- Liveness detection: blink-detection and/or head-turn challenge using MediaPipe face-mesh landmarks (eye-aspect-ratio for blink, yaw angle for head turn).
- Liveness challenge state machine (e.g., "turn your head left", then verify within N seconds) exposed via the internal API (§11).
- Matplotlib rendering: turn Member 3's aggregated data into actual chart images (bar/line/pie as appropriate) returned as PNG/base64 from a Python endpoint that Java calls for the "download report" flow.

Primary ownership — Frontend (React + TypeScript + Tailwind):

- Full React app scaffolding (Vite, routing, Axios client with JWT interceptor).
- Student pages: Login, Register, Dashboard, Active Sessions, Mark Attendance flow (camera + geolocation UI), Liveness Challenge UI, Attendance Result, History, Percentage, Profile.
- Teacher/Admin pages: Login, Dashboard, Course Management, Student Management, Create Session, Session Details, Live Attendance (WebSocket-driven), History, Analytics (Recharts), Reports, Profile.
- Recharts/Chart.js interactive dashboard visualizations (this is explicitly **not** Matplotlib — Matplotlib is only for the static/exported report images).
- Docker Compose file wiring all 4 containers (react/nginx, spring-boot, fastapi, postgres) — since you already own Dockerfiles conceptually across the stack, you're the natural integrator here, with Member 3 supplying the Python Dockerfile.
- CI/CD support (a simple GitHub Actions pipeline: build + test on push) and the top-level README/setup documentation.

Why this is the correct owner for dashboards even though you're a "Python developer": dashboards are a rendering/UX problem (React state, component composition, chart libraries), not a computer-vision problem. Putting them in Python would mean either duplicating a web framework in Python (unnecessary complexity) or building a second frontend stack — neither is justified here.

8. Responsibility Matrix

Capability	Java M1	Java M2	Python M3	Python M4
DB schema & migrations	✔ Owner	contributes (session/attendance tables)	-	-
Auth (JWT, RBAC)	✔ Owner	uses	-	uses (login UI)
Course / Enrollment	✔ Owner	-	-	UI only
Attendance session	-	✔ Owner	-	UI only
Geofence + Haversine	-	✔ Owner	-	sends GPS coords (UI)
Java→Python client	-	✔ Owner	provides contract	-
WebSocket	-	✔ Owner	-	consumes (live UI)
Face detection/embedding/match	-	-	✔ Owner	-
Liveness challenge	-	-	-	✔ Owner
Matplotlib data	-	-	✔ Owner (data)	✔ Owner (render/serve)
Recharts dashboards	-	-	-	✔ Owner
Excel/CSV reports	utility skeleton	✔ Owner	-	-
Security hardening/rate limit	-	✔ Owner	internal-only network	-
Docker (per-service)	-	-	Python image	Compose + wiring
Docs	SRS, install guide	DB/API rationale	ML methodology	README, setup

Who blocks whom (critical dependency chain):

1. Member 1's `User / Student` entities + JWT block *everyone* — must land Week 1.
2. Member 1 + Member 2 must **freeze the Python API contract together with Member 3** by end of Week 1 — this blocks Member 2's orchestration code and Member 3's endpoint shapes simultaneously.
3. Member 2's session/geofence endpoints block Member 4's "Mark Attendance" UI.
4. Member 3's face endpoints block Member 4's "Face Verification" UI and Member 2's orchestration logic.
5. Member 4's liveness endpoint blocks Member 2's orchestration logic (liveness runs before face match in the flow).
6. Member 2's WebSocket topic naming blocks Member 4's Live Attendance dashboard.

This is why §14's timeline front-loads the contract freeze and a stub-based Java build in Week 1–2, rather than letting Java and Python develop in isolation until Week 5.

9. Database Design (PostgreSQL)

All tables use `BIGSERIAL / UUID` primary keys (either is fine; examples below use `BIGSERIAL` for simplicity) and `created_at / updated_at` timestamps.

users

- PK: `id`
- Fields: `email` (unique, not null), `password_hash`, `role` (`STUDENT / TEACHER / ADMIN`), `full_name`, `is_active`
- Constraints: `UNIQUE(email)`
- Relationships: 1-1 with `students` or `teachers` depending on role

students

- PK: `id`
- FK: `user_id` → `users.id` (unique — one student profile per user)
- Fields: `roll_number` (unique), `department`, `year`
- Constraints: `UNIQUE(roll_number)`, `UNIQUE(user_id)`

teachers

- PK: `id`
- FK: `user_id` → `users.id` (unique)
- Fields: `department`, `designation`

courses

- PK: `id`
- FK: `teacher_id` → `teachers.id`
- Fields: `course_code` (unique), `title`, `semester`
- Constraints: `UNIQUE(course_code)`

enrollments

- PK: `id`
- FK: `student_id` → `students.id`, `course_id` → `courses.id`
- Constraints: `UNIQUE(student_id, course_id)` — a student can't enroll twice in the same course
- Fields: `enrolled_at`

face_enrollments

- PK: `id`
- FK: `student_id` → `students.id` (unique — one active embedding per student, or versioned if you allow re-enrollment)
- Fields: `embedding` (stored as `bytea` or `float[]` / vector if using `pgvector` — plain `bytea` of a serialized float array is enough for a college project), `embedding_version`, `enrolled_at`, `quality_score`
- Constraints: `UNIQUE(student_id)` (or drop uniqueness + add `is_active` boolean if re-enrollment history is desired)
- Note: never store the raw enrollment photo in the DB long-term — store only the embedding (see §16 security).

attendance_sessions

- PK: `id`
- FK: `course_id` → `courses.id`, `created_by` → `teachers.id`
- Fields: `latitude`, `longitude`, `radius_meters`, `start_time`, `end_time`, `status` (`SCHEDULED` / `ACTIVE` / `ENDED`)
- Constraints: `CHECK(radius_meters > 0)`, `CHECK(end_time > start_time)`

attendance

- PK: `id`
- FK: `student_id` → `students.id`, `session_id` → `attendance_sessions.id`
- Fields: `marked_at`, `distance_meters`, `face_match_score`, `liveness_passed` (boolean), `status` (`PRESENT`)
- **Constraints:** `UNIQUE(student_id, session_id)` — this is the DB-level duplicate-attendance guard required in the brief; even if application logic has a bug or a race condition occurs under concurrent requests, Postgres itself rejects the second insert.

verification_attempts

- PK: `id`
- FK: `student_id` → `students.id`, `session_id` → `attendance_sessions.id`
- Fields: `attempt_type` (`GEOFENCE` / `LIVENESS` / `FACE`), `result` (boolean), `score` / `distance`, `reason` (e.g. `"out_of_radius"`, `"low_confidence"`, `"liveness_failed"`), `attempted_at`
- Purpose: keeps a full audit trail of every attempt (including failed ones) separately from the final `attendance` record — useful for debugging and for the "failed attempt" analytics chart.

audit_logs

- PK: `id`
- FK: `user_id` → `users.id` (nullable, for system events)
- Fields: `action`, `entity_type`, `entity_id`, `metadata` (`jsonb`), `ip_address`, `created_at`
- Purpose: security/traceability — who created a course, who started a session, who exported a report, any failed login, etc.

Relationship summary: `users` 1–1 `students/teachers`, `teachers` 1–N `courses`, `courses` N–N `students` (via `enrollments`), `courses` 1–N `attendance_sessions`, `attendance_sessions` 1–N `attendance`, `students` 1–1 `face_enrollments`, `students/sessions` 1–N `verification_attempts`.

10. Java API List (public — the only surface the browser touches)

Auth (Member 1)

- `POST /api/auth/register` → create user (student/teacher) → returns success
- `POST /api/auth/login` → validate credentials → returns JWT + refresh token
- `GET /api/auth/profile` → returns current user profile (JWT-protected)

Courses (Member 1)

- `POST /api/courses` (teacher/admin) → create course
- `PUT /api/courses/{id}` → update
- `DELETE /api/courses/{id}` → delete
- `GET /api/courses` → list (filtered by teacher, or all for admin)

Enrollment (Member 1)

- `POST /api/courses/{id}/enroll` (body: `studentId`) → enroll student
- `DELETE /api/courses/{id}/enroll/{studentId}` → remove
- `GET /api/courses/{id}/students` → list enrolled students

Sessions (Member 2)

- `POST /api/sessions` → create session (course, lat/lng, radius, start/end)
- `POST /api/sessions/{id}/start` → mark ACTIVE
- `POST /api/sessions/{id}/end` → mark ENDED
- `GET /api/sessions/active` → sessions currently active for the logged-in student's enrolled courses

Verification (Member 2, calling into Python via \$11)

- `POST /api/verification/location` → body: `{sessionId, lat, lng, accuracy}` → Haversine check → pass/fail + distance
- `POST /api/verification/liveness` → body: `{sessionId, imageFrames[]}` → proxies to Python liveness endpoint → pass/fail
- `POST /api/verification/face` → body: `{sessionId, image}` → proxies to Python face-match endpoint → match/no-match + score

Attendance (Member 2)

- `POST /api/attendance/mark` → body: `{sessionId}` (called only after all three verifications pass in sequence) → inserts row, triggers WebSocket broadcast
- `GET /api/attendance/history` → student's own history
- `GET /api/attendance/percentage/{courseId}` → computed attendance %

Reports (Member 2, built on Member 1's utility)

- `GET /api/reports/session/{id}/csv`
- `GET /api/reports/session/{id}/excel`

Admin/Teacher (split: CRUD parts by Member 1, live/session parts by Member 2)

- `GET /api/admin/students`, `GET /api/admin/courses` (Member 1)
- `GET /api/sessions/{id}/live` (WebSocket-backed live attendance list) (Member 2)

11. Python API Contract (internal only — frozen Week 1, never renegotiated mid-project without both sides agreeing)

This service sits behind Spring Boot only. In `docker-compose.yml` it has **no ports: mapping to the host** — only Spring Boot's service name resolves it on the internal Docker network.

Endpoint	Method	Request	Response	Purpose	Implements	Called by (Java)
POST /internal/face/enroll	POST	{studentId, image(base64)}	{success, qualityScore, embeddingId}	Generate + store embedding at student registration time	Member 3	Student profile/enrollment service
POST /internal/face/verify	POST	{studentId, image(base64), sessionId}	{match: bool, confidence: float, reason?: string}	Compare live capture against stored embedding	Member 3	Verification service (/api/verification/face)
POST /internal/liveness/start	POST	{sessionId, studentId}	{challengeId, instruction: "blink"}	"turn_left"	Issue a randomized liveness challenge	Member 4
POST /internal/liveness/verify	POST	{challengeId, imageFrames(base64[])}	{passed: bool, reason?: string}	Verify the challenge was completed live	Member 4	Verification service (/api/verification/liveness)
GET /internal/reports/chart	GET	{sessionId or courseId, chartType}	{imageBase64} (Matplotlib PNG)	Generate a static analytics chart image	Member 3 (data) + Member 4 (render)	Reports service, for the "download visual report" option

Design notes:

- Face detect + embed + compare are **combined into two endpoints** (`enroll` , `verify`) rather than four separate ones (`detect` , `embed` , `compare` as separate calls) — because Java never needs the intermediate detection/embedding result on its own; it always wants a final yes/no + score. Fewer network hops, fewer partial-failure states to handle in Java. Internally in the Python code, detection/quality-check/embedding/comparison remain separate functions/modules for testability — the API surface is just consolidated.
- Liveness is kept as **two calls** (`start` , `verify`) instead of one, because a challenge is issued, the student performs an action over a few seconds, *then* frames are verified — it's inherently two round trips.
- All image payloads are base64 JPEG, capped at a sane size (e.g. 2MB) validated on both sides.
- Every response includes a `reason` field on failure (`"no_face_detected"` , `"multiple_faces"` , `"low_confidence"` , `"blurry_image"` , `"liveness_timeout"`) so Java can show a specific, helpful error in the UI instead of a generic failure.

12. React Pages

Student | Page | Purpose | Key components | API used | Owner | |---|---|---|---|---| | Login | Auth | Form, error toast | `/api/auth/login` | M4 | | Register | Signup | Form + validation | `/api/auth/register` | M4 | | Dashboard | Overview | Stat cards, upcoming sessions | `/api/sessions/active` , `/api/attendance/percentage` | M4 | | Active Sessions | List joinable sessions | Session cards | `/api/sessions/active` | M4 | | Mark Attendance | Orchestrates the full flow | Geolocation prompt → camera → progress steps | `/api/verification/location` , `/liveness` , `/face` , `/api/attendance/mark` | M4 | | Location Permission | Request/explain GPS access | Permission modal | Browser Geolocation API | M4 | | Liveness Challenge | Show instruction, capture frames | Camera + instruction overlay + timer | `/api/verification/liveness` | M4 | | Face Verification | Capture + submit face | Camera capture component | `/api/verification/face` | M4 | | Attendance Result | Success/fail screen | Result card, reason message | (uses result from mark flow) | M4 | | Attendance History | Past records | Table/list | `/api/attendance/history` | M4 | | Attendance Percentage | Per-course % | Progress bars, Recharts | `/api/attendance/percentage/{id}` | M4 | | Profile | View/edit profile, face re-enrollment entry point | Form | `/api/auth/profile` | M4 |

Teacher/Admin | Page | Purpose | Key components | API used | Owner | |---|---|---|---|---| | Login | Auth | Form | `/api/auth/login` | M4 | | Dashboard | Overview | Stat cards | course/session summary APIs | M4 | | Course Management | CRUD courses | Table + modal form | `/api/courses` | M4 | | Student Management | View/enroll students | Table + search + enroll modal | `/api/courses/{id}/students` | M4 | | Create Attendance Session | Set location/radius/time | Map picker (or manual lat/lng), radius slider, datetime pickers | `/api/sessions` | M4 | | Session Details | Single session view | Info card | `/api/sessions/{id}` | M4 | | Live Attendance | Real-time list as students check in | WebSocket-fed table | STOMP subscribe `/topic/session/{id}` | M4 (UI) + M2 (socket contract) | | Attendance History | Past sessions' records | Table + filters | `/api/reports` , attendance list APIs | M4 | | Analytics | Interactive charts | Recharts bar/line/pie | attendance aggregate APIs | M4 | | Reports | Download CSV/Excel/PNG | Buttons | `/api/reports/.../csv` , `/excel` , chart endpoint | M4 | | Profile | Teacher profile | Form | `/api/auth/profile` | M4 |

(All frontend pages are owned by Member 4 by design — this is the balancing trade-off for Member 4 also owning Python liveness; see §22.)

13. Complete Attendance Flow — who owns every step

1. **Teacher login** → Java auth (M1) → React login page (M4)
2. **Create course** → Java course API (M1) → React Course Management page (M4)
3. **Create attendance session, set location/radius/time** → Java session API (M2) → React Create Session page (M4)
4. **Start session** → Java session API (M2)
5. **Student login** → Java auth (M1) → React (M4)
6. **Student views active session** → Java `/api/sessions/active` (M2) → React Active Sessions page (M4)
7. **Student allows location** → Browser Geolocation API in React (M4)
8. **React sends GPS coords to Java** → `/api/verification/location` (M2)
9. **Java verifies geofence** (Haversine, radius check) → M2
10. **Java (via M2's client) starts liveness challenge** → calls Python `/internal/liveness/start` (M4's endpoint)
11. **React shows liveness instruction, captures frames** → M4 (frontend)
12. **Java sends frames to Python** → `/internal/liveness/verify` (M4's endpoint) → M2's Java client
13. **Python verifies liveness** → M4 (Python logic)
14. **React captures face image** → M4 (frontend camera component)
15. **Java sends image to Python** → `/internal/face/verify` (M3's endpoint) → M2's Java client
16. **Python detects face, generates embedding, compares to enrolled embedding** → M3
17. **Python returns match + confidence** → M3
18. **Java validates all three results together** (geofence pass + liveness pass + face match $\geq$ threshold) → M2
19. **Attendance marked** (insert into `attendance`, protected by unique constraint) → M2
20. **WebSocket broadcasts update** → M2 (backend), consumed by React → M4 (Live Attendance page)
21. **Teacher dashboard updates instantly** → M4 (frontend), M2 (socket contract)

14. 6-Week Timeline

Week 1 — Foundation & Contracts

- M1: Project skeleton, DB schema v1 (migrations), User/Student/Teacher entities, JWT skeleton
- M2: Session/Attendance entity design (paired with M1 on schema), draft geofence algorithm, **co-author Python API contract with M3/M4**
- M3: FastAPI skeleton, pick and test an embedding model, first working detect+embed script (CLI, no API yet)
- M4: React project scaffold (routing, Tailwind, Axios), Docker Compose skeleton, liveness proof-of-concept script
- **Integration task:** All 4 agree and commit `API_CONTRACT.md` (Python endpoints frozen) + Postman collection stub
- **Milestone:** Repo structure + schema + frozen contract + everyone can run `docker compose up` with empty services
- **Testing:** Schema review, contract review (does every field needed downstream exist?)

Week 2 — Auth + Stubbed Integration

- M1: Finish auth (register/login/profile), RBAC, JWT filter, Course/Enrollment CRUD
- M2: Session CRUD (`create/start/end/active`), start Java→Python client **against a mock Python server** (simple stub returning fixed JSON matching the contract)
- M3: Real `/internal/face/enroll` and `/internal/face/verify` working end-to-end in FastAPI (not yet perfect accuracy)
- M4: Login/Register pages wired to real Java auth; Dashboard shell; start liveness FastAPI endpoint
- **Integration task:** Replace Java's mock Python stub with M3's real face endpoints for the enroll flow
- **Milestone:** A user can register, login, and a teacher can create a course — end-to-end through real Java + real React
- **Testing:** JUnit for auth/course; Postman smoke tests for all Week 1-2 endpoints

Week 3 — Sessions, Geofence, Face Enrollment, Dashboards

- M1: Polish DTO validation, exception handling across modules, support M2 on Excel utility skeleton
- M2: Geofence/Haversine verification fully implemented and unit-tested; `/api/verification/location` live
- M3: Face quality checks (blur/lighting), threshold tuning begins with sample photos from the team
- M4: Face enrollment UI (Profile page), Create Session page, Student/Teacher dashboard shells with mock data
- **Integration task:** Face enrollment flow working fully: React captures photo → Java → Python `/internal/face/enroll` → DB row in `face_enrollments`
- **Milestone:** A student can enroll their face and a teacher can create a geofenced session
- **Testing:** Geofence unit tests (in-radius/out-of-radius edge cases), face enrollment manual test with real photos

Week 4 — Full End-to-End Attendance Flow (critical week)

- M1: Support/bugfix as needed; begin admin student-management pages backend support
- M2: Wire the full orchestration: geofence → liveness → face → mark attendance, in sequence, with proper error propagation
- M3: `/internal/face/verify` accuracy pass — test with multiple people, tune threshold
- M4: Full Mark Attendance UI flow (location permission → liveness challenge → face capture → result screen); liveness challenge logic finished in Python
- **Integration task:** Whole-team session — run the *entire* flow live (steps 1-19 of §13) with real devices/webcams

- **Milestone:** A student can walk through the full attendance flow and see a real PRESENT/REJECTED result
- **Testing:** End-to-end manual test script covering happy path + at least 3 failure paths (out of radius, face mismatch, liveness fail)

Week 5 — Real-time, Analytics, Reports, Security, Edge Cases

- M1: Security review pass (input validation, CORS config finalization), audit_logs implementation
- M2: WebSocket live broadcast; Excel/CSV report endpoints; rate limiting on verification endpoints; duplicate-attendance test at DB level
- M3: Matplotlib data-prep functions (confidence distributions, per-session pass/fail counts); edge case hardening (no face, multiple faces, blurry image all return clean `reason` codes)
- M4: Live Attendance dashboard (WebSocket-fed), Analytics page with Recharts, Reports page (CSV/Excel/chart download), Matplotlib chart image integration
- **Integration task:** Live Attendance page verified against real WebSocket broadcasts from concurrent "student" sessions
- **Milestone:** Teacher sees attendance appear on screen in real time; downloadable Excel/CSV/PNG reports work
- **Testing:** WebSocket load test with 5-10 simulated students marking attendance in the same session; report file validation (open in Excel, check row counts)

Week 6 — Testing, Bug Fixing, Docker, Documentation, Demo Prep

- M1: Finalize JUnit coverage on core modules, SRS + installation guide docs, help fix cross-cutting bugs
- M2: Finalize integration tests, security hardening review, help fix bugs in attendance flow
- M3: Finalize ML unit tests, ML methodology doc, help fix Python-side bugs
- M4: Full Docker Compose validation (`docker compose up` from a clean machine), CI pipeline green, README/setup guide, rehearse demo script (§20)
- **Integration task:** Two full dry-run demos on a machine that has never run the project before (fresh clone → `docker compose up` → full flow)
- **Milestone:** Demo-ready build, tagged `v1.0`, all docs committed
- **Testing:** Full regression pass against the demo script in §20; fix anything that breaks

15. Git/GitHub Strategy

- **Branching:** `main` (protected, always deployable) ← `develop` ← feature branches named `feature/<member>-<short-desc>` (e.g. `feature/m2-geofence-haversine`).
- **Pull requests:** every feature branch → PR into `develop`, requires at least 1 reviewer (pair Java devs review each other's Java PRs, Python devs review each other's Python PRs; cross-review encouraged for the integration-critical pieces like the Java-Python client and the API contract).
- **API contract as a living doc:** `API_CONTRACT.md` (or an OpenAPI/Swagger spec) checked into the repo root at Week 1, changed only via PR with both a Java and a Python reviewer.
- **Mock APIs / stubs:** Member 2 builds a tiny stub FastAPI-shaped mock (or uses WireMock/JSON-server) returning contract-shaped fixed responses, so Java development on the orchestration service is never blocked waiting for Member 3/4's real ML code — this is how "Java develops against a Python stub" works concretely (see Week 2 above).
- **Postman:** one shared Postman collection per service (Java public API, Python internal API), exported to the repo as JSON so it's version-controlled, not just living in someone's personal workspace.
- **Environment variables:** every service has a `.env.example` (never commit real `.env`) listing `DB_URL`, `JWT_SECRET`, `PYTHON_SERVICE_URL`, `FACE_MATCH_THRESHOLD`, etc.
- **Docker Compose:** one root `docker-compose.yml` bringing up `postgres`, `spring-boot`, `fastapi`, `react` (served via nginx or Vite preview) — this is the thing every member runs locally to test against the real stack once services stabilize.
- **Shared test data:** a `seed.sql` (owned by M1) with a handful of test users/courses/sessions so everyone's local DB looks the same; a small shared folder of sample face photos (owned by M3) for manual testing of enrollment/verification.
- **Integration checkpoints:** end of Week 1 (contract freeze), end of Week 2 (auth+stub), end of Week 4 (full E2E flow), end of Week 5 (real-time+reports) — each checkpoint is a whole-team working session, not left to chance.

16. Testing Strategy

Java (M1 primarily on auth/user/course, M2 on session/attendance/geofence)

- JUnit + Mockito for service-layer unit tests
- Controller tests (MockMvc) for request/response contracts
- Security tests: unauthorized access rejected, role checks enforced, expired/invalid JWT rejected
- Geofence tests: exact boundary (distance == radius), just inside, just outside, invalid coordinates
- Attendance duplicate test: attempt to insert two `attendance` rows for the same `(student_id, session_id)` and assert the DB constraint throws

Python (M3 on face, M4 on liveness)

- Face detection tests: known face detected, no face → clean rejection, multiple faces → clean rejection
- Embedding tests: same person → high similarity, different people → low similarity
- Face matching threshold tests: values just above/below the configured threshold
- Liveness tests: genuine live sequence passes, static photo replay fails, video replay attempt fails
- Robustness: poor lighting, blurry image, partial face — all return a specific `reason`, never a 500 error

Frontend (M4)

- Form validation (empty fields, invalid email, password rules)
- Permission handling: camera denied, location denied — both show a clear recoverable message, not a blank screen
- API error handling: expired token → redirect to login; 4xx/5xx → toast with retry option
- Camera/location error simulation (deny permission in browser devtools)
- Responsive UI check on mobile viewport (since students will likely use phones for GPS+camera)

Integration (whole team, at the checkpoints in §14/§15)

- Java ↔ Python: contract compliance tests (send a request per §11, assert response shape)
- React ↔ Java: full auth-to-attendance flow via Cypress/Playwright or manual scripted test
- Database: constraint tests (duplicate attendance, cascade behavior on delete)
- WebSocket: multiple simulated clients receive the broadcast in real time
- Complete attendance flow: the full 21-step chain in §13, run manually at least twice per week from Week 4 onward

17. Security Strategy

Real security features (needed and implemented):

- JWT-based stateless auth with expiry + refresh
- BCrypt password hashing (never plaintext, never reversible encoding)
- Role-based method-level authorization (`@PreAuthorize`)
- Input validation on every DTO (Bean Validation) — rejects malformed lat/lng, oversized images, etc.
- CORS restricted to the known frontend origin
- Rate limiting on verification endpoints (prevents brute-force face-match attempts / GPS spoofing spam)
- Environment variables for all secrets (JWT secret, DB credentials) — never hardcoded, never committed
- Python service is **internal-only** — not reachable from the browser or the internet, only from the Spring Boot container on the Docker network
- Face embeddings are never returned in any API response to the browser (only match/no-match + score go back)
- Raw enrollment/verification photos are not persisted after processing (only the embedding is stored) — reduces biometric data exposure
- Audit log for sensitive actions (login attempts, session creation, report export)
- DB-level unique constraint prevents duplicate attendance even under concurrent requests (a real security/integrity feature, not just a nicety)

"Nice to mention but basic college-project level" (be upfront about this in your report/demo Q&A):

- Embedding storage is not hardware-encrypted (e.g., no HSM/KMS) — for production you'd encrypt at rest with a managed key service; here, storing it as `bytea` in Postgres with disk-level encryption (if your deployment environment provides it) is "good enough for a student project," not enterprise-grade biometric protection.
- Liveness detection (blink/head-turn) is a reasonable anti-spoofing measure but does **not** defeat a sophisticated deepfake/replay attack — that's an accepted limitation, not a gap you need to solve (see §18 risks).
- Rate limiting is in-memory/simple (e.g., Bucket4j single-node), not a distributed rate limiter — fine for a single-instance college deployment, not for production scale.
- No formal penetration testing — you're doing test-driven security checks (JUnit "unauthorized access rejected" tests), which is appropriate scope for this project.

18. Risk Management

Risk	Impact	Prevention	Backup Solution
dlib/OpenCV install issues (native deps, wheel build failures)	Blocks all Python CV work	Use a Docker base image with dependencies pre-baked; pin exact versions in <code>requirements.txt</code> / <code>Dockerfile</code> early (Week 1)	Fall back to a pure-Python embedding library (e.g., <code>face_recognition</code> prebuilt wheel, or a smaller ONNX model) that doesn't need a compiled dlib build
Camera permission denied/unavailable	Student can't mark attendance	Clear UI messaging, permission-request flow tested on Chrome/Firefox/mobile	Manual/admin override path for edge cases (teacher can manually mark, logged in audit log)
GPS accuracy indoors (multi-story buildings, weak signal)	False geofence rejections	Use <code>accuracy</code> field from Geolocation API; allow slightly generous radius; document as a known limitation	Allow teacher to widen radius per session, or a manual admin approval fallback for borderline cases
Poor lighting for face capture	False rejections	Quality check step (brightness check) with a clear retry prompt in UI	Retry mechanism with up to N attempts before showing "contact teacher"
Multiple faces in frame	Ambiguous verification	Explicit multi-face detection returns clean rejection reason	UI instructs "make sure only your face is visible," then retry
Face mismatch (false negative on legitimate student)	Student wrongly denied	Careful threshold tuning in Week 3 with real test data from the team	Manual teacher override to mark present with a logged justification
Liveness detection limitations (defeatable by sophisticated spoofing)	Security gap (accepted)	Document clearly as a known limitation in Limitations (§19)	Not attempting to solve — out of scope for a college project
Python↔Java API mismatch	Integration breaks late	Contract frozen Week 1, changes only via reviewed PR, stub-based Java dev in Week 2	Contract version field in requests/responses if a breaking change becomes unavoidable
Database race conditions (concurrent attendance marks)	Duplicate/incorrect rows	DB-level <code>UNIQUE(student_id, session_id)</code> constraint (not just app-level check)	Catch the constraint violation exception in Java and return a friendly "already marked" response
WebSocket connection issues (drops, reconnect)	Live dashboard stalls	STOMP client auto-reconnect logic in React	Fallback: manual refresh/poll button on the Live Attendance page
Docker setup problems across team machines	Wasted setup time	<code>docker-compose.yml</code> + <code>.env.example</code> finalized and tested by Week 1, README with exact commands	Each member also documents a "run without Docker" fallback for their own service
Unequal workload emerging mid-project	Team friction, missed deadlines	Weekly check-in against this plan's task list; redistribute secondary tasks (not core ownership) as needed	Rebalancing already built into §22 — revisit it explicitly at the Week 3 checkpoint

19. Documentation Division

Document	Owner
SRS (Software Requirements Spec)	M1
System Architecture diagram/writeup	M2 (co-reviewed by all)
Use Case diagram	M1
Class diagram (Java side)	M1
ER diagram	M1
Sequence diagrams (attendance flow, per §13)	M2
Activity diagram	M2
API documentation (Java public API)	M1 (auth/course/enrollment) + M2 (session/attendance/reports)
API documentation (Python internal contract)	M3 (face) + M4 (liveness/reports)
Database documentation	M1 (schema) + M2 (rationale for session/attendance tables)
Testing documentation	M2 (Java/integration) + M3 (ML tests) + M4 (frontend tests)
Security documentation	M1 (auth/RBAC) + M2 (rate limiting/network isolation)
ML methodology (face + liveness approach, threshold choice)	M3 (face) + M4 (liveness)
Geofencing methodology (Haversine explanation, radius rationale)	M2
Screenshots (all pages, all flows)	M4
Installation guide	M1 (drafts) + M4 (Docker-specific steps, finalizes)
Limitations	M3 (ML limitations) + M2 (geofence/GPS limitations) — combined by M4 into final doc
Future scope	Whole team — 30-min group brainstorm in Week 6, written up by M4

20. Final Demo Flow (exact sequence)

- Admin/Teacher logs in (M1's auth)
- Teacher creates a course (M1's course API)
- Teacher enrolls students into the course (M1's enrollment API)

4. Teacher creates an attendance session, sets location + radius (M2's session API)
5. Teacher starts the session
6. Student logs in on a separate device/browser (M1's auth)
7. Student sees the active session on their dashboard (M2's session API)
8. Student allows location → GPS sent to Java → geofence verified live on screen (M2)
9. Liveness challenge appears, student completes it (M4 Python + UI)
10. Face verification: student's camera captures and verifies against enrolled face (M3 + M4 UI)
11. Attendance marked — success screen shown (M2)
12. Teacher's Live Attendance dashboard updates instantly via WebSocket, in front of the audience (M2 backend + M4 frontend)
13. Switch to Analytics page — show interactive Recharts graphs (M4)
14. Download/show a Matplotlib-generated report graph (M3 data + M4 render)
15. Export and open the Excel/CSV report (M2)
16. **Duplicate attempt:** the same student tries to mark attendance again for the same session → system cleanly rejects it (demonstrates the DB unique constraint) (M2)
17. **Unauthorized API call:** show a raw API call (e.g., via Postman) to a protected endpoint without a valid JWT, or a student role hitting a teacher-only endpoint → rejected with 401/403 (M1)

21. Final Acceptance Checklist

- ☐ Student can register, enroll face, and login
- ☐ Teacher can create course, enroll students, create/start/end a geofenced session
- ☐ GPS coordinates are sent from React to Java only (never directly to Python)
- ☐ Java correctly computes Haversine distance and rejects out-of-radius attempts
- ☐ Liveness challenge runs and correctly rejects a static photo held to the camera
- ☐ Face verification correctly matches the enrolled student and rejects a different person
- ☐ Attendance is marked only when geofence + liveness + face all pass
- ☐ Duplicate attendance for the same `(student, session)` is rejected at the DB level
- ☐ Teacher's Live Attendance dashboard updates in real time via WebSocket
- ☐ Analytics page shows interactive Recharts/Chart.js visualizations
- ☐ A Matplotlib-generated chart image can be downloaded/viewed
- ☐ CSV and Excel reports export correctly and open without corruption
- ☐ Unauthorized/unauthenticated API calls are rejected
- ☐ The Python service is not reachable from outside the Docker network
- ☐ `docker compose up` brings up the entire system on a clean machine
- ☐ All core JUnit/Python/frontend tests pass
- ☐ All required documentation (§19) is complete

22. Suggestions for Improving Team Balance

Raw ownership (ignoring secondary tasks) looks like this before rebalancing:

- **M1** — broad but shallow: 6 CRUD-ish modules, foundational but individually simple.
- **M2** — narrow but deep: geofencing math + the single most complex piece of glue code in the system (orchestration) + WebSocket + reports + security.
- **M3** — deep and research-heavy: the core ML accuracy work, which tends to eat unpredictable time (threshold tuning is trial-and-error).
- **M4** — two full jobs stacked: Python liveness *and* practically the entire frontend (12+ pages).

Rebalancing already applied in this plan:

1. Member 1 was given the shared `ExcelExportUtil` skeleton and exception-handling framework so Member 2 doesn't start report/error-handling work from zero — this shifts some "plumbing" hours from M2 to M1 without changing who *owns* reports.
2. Member 3 was given the Matplotlib **data preparation** work (not the rendering), splitting that line item with Member 4 instead of leaving it entirely on M4.
3. Member 3 was given ownership of the Python `Dockerfile` (the person who fights dlib/OpenCV installs should own that artifact), taking one Docker task off Member 4's very full plate.
4. Member 1 was given SRS + Installation Guide (drafts) to offset Member 2 and Member 4 carrying heavier documentation loads elsewhere (sequence diagrams, ML methodology, screenshots).

If Week 3's checkpoint shows Member 4 still overloaded (likely, given 12+ React pages), the recommended further adjustment is:

- Move the **Teacher/Admin "Student Management" and "Course Management" pages** (relatively simple CRUD-table UIs) to be built jointly with Member 1 doing basic scaffolding/wireframe-level React (Member 1 doesn't need to be a React expert to build a simple table + modal form) — Member 4 then only polishes styling. This is optional and should only trigger if the Week 3 check-in shows a real imbalance, not applied by default.
- Alternatively, move the **CSV/Excel report UI buttons** (not the backend) from Member 4's Reports page into Member 2's integration testing week, since Member 2 already deeply understands the report data shape.

General practice for the team: do a 15-minute workload check-in at the end of each week against this plan. If any member is more than ~1 day behind the others by Week 3, redistribute a **secondary** task (never a primary ownership item — that would break the clean architecture boundaries this plan relies on) using the options above.